

Introduction to Machine Learning

Lecture 12: Representation via Network of Classifiers

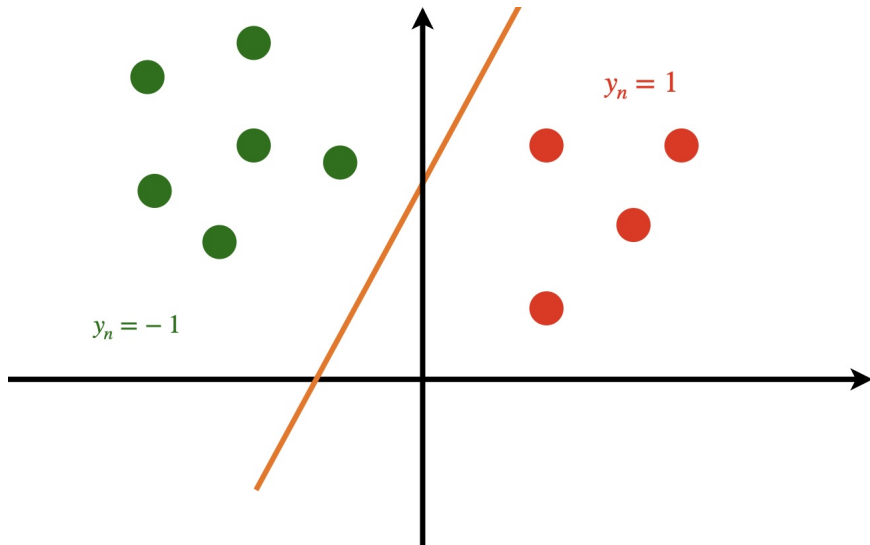
Ali Bereyhi

`ali.bereyhi@utoronto.ca`

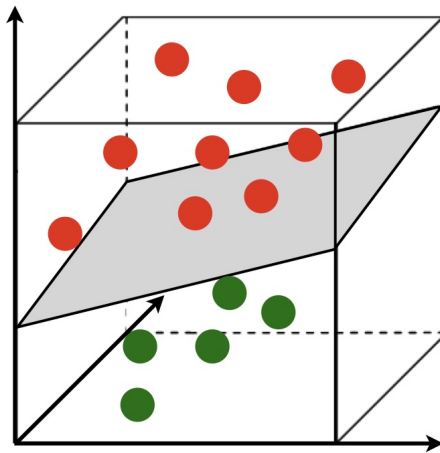
Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Quick Recap: *Linear Classifier*



Quick Recap: *Support Vector Machine*



Today's Agenda: *Support Vector Machine*

Today, we study a class of nonlinear models which is very powerful, i.e.,

Neural Networks

In this way, we discuss the following topics

- *Representing a binary function*
- *Neural networks as universal representation*
- *Expressive power of neural networks*
 - ↳ *Universal Approximation Theorem*

Learning Binary Functions

We are given with all four cases of two binary variables ^{samples} and ~~their OR~~ ^{labels}, i.e.,

$$\mathbf{x} = [x_1, x_2] \quad \mathbb{D} = \{([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 1)\}$$

$v = x_1 \vee x_2$

Question: Can we learn this binary function by a linear classifier from $\mathbb{D}$?

$$y = \mathbf{w}^T \mathbf{x} + b \rightsquigarrow v = \begin{cases} 1 & y \geq 0 \\ 0 & y < 0 \end{cases}$$

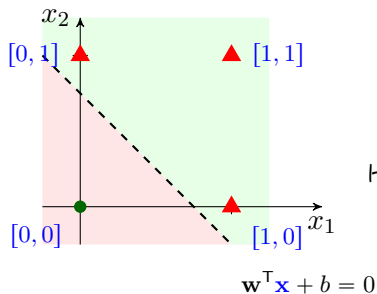
Example: OR Function

The OR of two binary variables $x_1, x_2 \in \{0, 1\}$ is

$$x_1 \vee x_2 = \begin{cases} 1 & \text{if } x_1 \text{ or } x_2 = 1 \\ 0 & \text{if } x_1 = x_2 = 0 \end{cases}$$

OR Function: *Binary Classification*

Let's show data-points with label $y = 1$ by ▲ and those with label $y = 0$ by ●

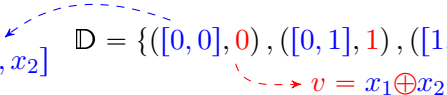


It works!

Learning XOR Function

Let's now consider the XOR example

$$\mathbf{x} = [x_1, x_2] \quad \mathbb{D} = \{([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 0)\}$$


 $v = x_1 \oplus x_2$

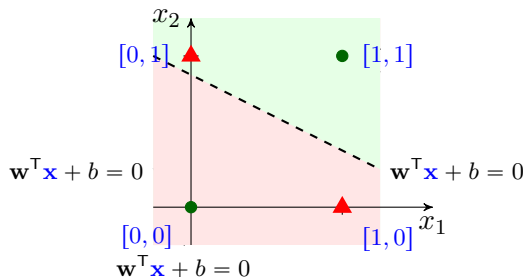
Reminder: XOR Function

The XOR of two binary variables $x_1, x_2 \in \{0, 1\}$ is

$$x_1 \oplus x_2 = \begin{cases} 1 & \text{if } x_1 \neq x_2 \\ 0 & \text{if } x_1 = x_2 \end{cases}$$

Learning XOR Function: Binary Classification

Let's show data-points with label $y = 1$ by $\blacktriangle$ and those with label $y = 0$ by $\bullet$

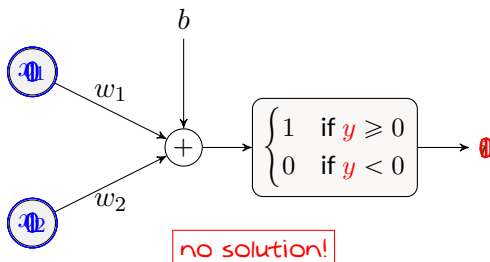


It seems to be impossible!
Well! We can show it

and look for $\mathbf{w}^T \mathbf{x} + b = 0$ that separates the labels

Learning XOR Function: Binary Classification

Let's check it $\mathbb{D} = \{([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 0)\}$



$$0w_1 + 0w_2 + b < 0 \rightsquigarrow b < 0$$

$$-2b \leq w_1 + w_2 < -b$$

$$0w_1 + 1w_2 + b \geq 0 \rightsquigarrow w_2 + b \geq 0$$

$$1w_1 + 0w_2 + b \geq 0 \rightsquigarrow w_1 + b \geq 0$$

$$1w_1 + 1w_2 + b < 0 \rightsquigarrow w_1 + w_2 + b < 0$$

$$\rightsquigarrow -2b \leq w_1 + w_2$$

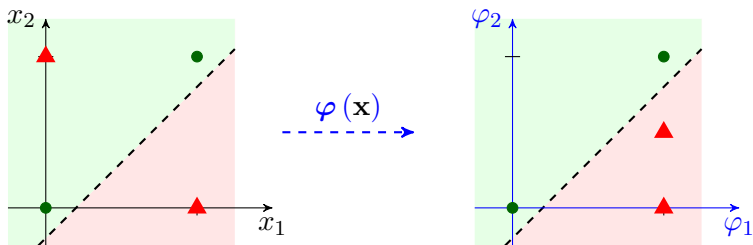
$$w_1 + w_2 < -b$$

Learning XOR Function: Simple Remedy

So, we can conclude that

*XOR function is **not** linearly separable*

How can we solve this problem? *SVM, and no need for higher dimensions!*



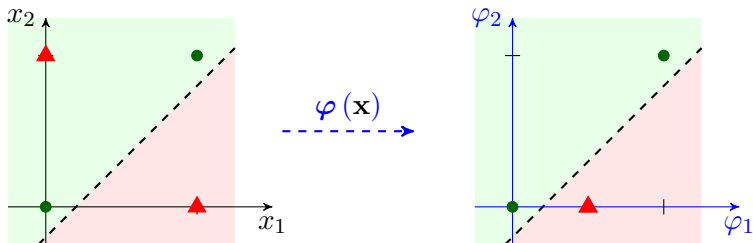
Then, we could learn XOR **perfectly** from $\varphi(\mathbf{x})$ via a linear model

Learning XOR Function: Simple Remedy

Consider the following transform

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \xrightarrow{\varphi(\mathbf{x})} \varphi(\mathbf{x}) = \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} = \begin{bmatrix} 0.5 \max \{x_1 + x_2, 0\} \\ \max \{x_1 + x_2 - 1, 0\} \end{bmatrix}$$

Let's apply the transform on our data points

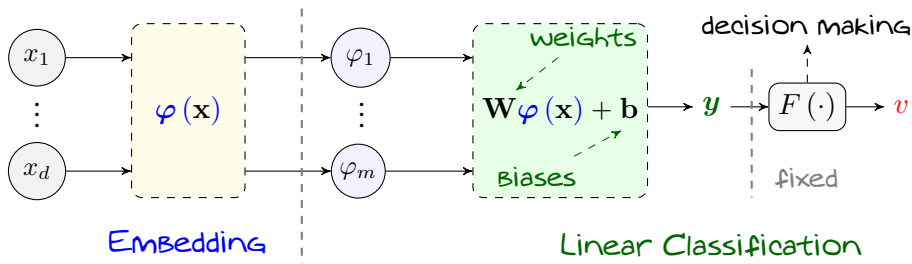


Applying binary classification on $\varphi(\mathbf{x})$, we can now learn XOR correctly

Visualizing the Nonlinear Model

We can visualize this as follows

How can we find the right embedding?



Learning Kernel

Attempt 1: Engineering

Engineer the feature, i.e., *find a good kernel by hand (trial and error)* ✕

Attempt 2: Representation Learning

Why not learning the *embedding* itself? Agree on a $\varphi(\mathbf{x}; \omega)$, e.g.,

$$\varphi(\mathbf{x}; \omega_0, \omega_1, \omega_2) = \omega_0 + \omega_1 \mathbf{x} + \omega_2 \mathbf{x}^2$$

Then try to learn ω along with the linear model, i.e., solve

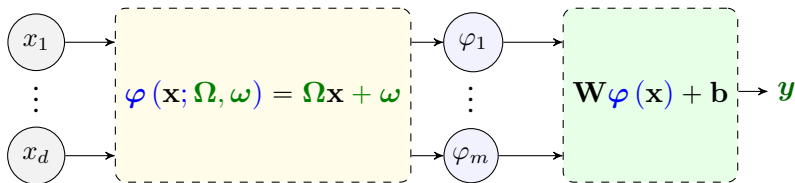
$$\min_{\mathbf{W}, \mathbf{b}} \hat{R}(\mathbf{W}, \mathbf{b}) \rightsquigarrow \min_{\mathbf{W}, \mathbf{b}, \omega} \hat{R}(\mathbf{W}, \mathbf{b}, \omega)$$

Basic Property of Embedding

A Basic Question

What kind of property should the **embedding** have?

Let's try a **naïve** choice, i.e., a *linear embedding*



In this **embedding**,

- dimension of ω , i.e., m , is a **hyper-parameter**
- Ω and ω are **learnable**

Basic Property of Kernels

Well, let's see how y looks like

$$\begin{aligned}
 y &= \mathbf{W}\varphi(\mathbf{x}) + \mathbf{b} = \mathbf{W}(\Omega\mathbf{x} + \omega) + \mathbf{b} = \cancel{\mathbf{W}\Omega}\mathbf{x} + \cancel{\mathbf{W}\omega} + \mathbf{b} \\
 &= \tilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}} \quad \text{linear embedding doesn't do anything!}
 \end{aligned}$$

A Basic Question

What kind of property should the **embedding** have?

Simple Answer

*It should definitely contain some **non-linearity***

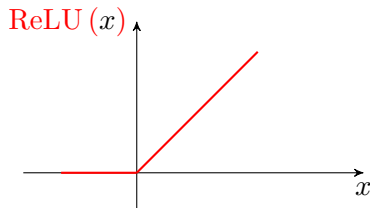
Wait a minute! Was it the case in the XOR example?!

Back to the XOR Example

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \xrightarrow{\varphi(\mathbf{x})} \varphi(\mathbf{x}) = \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} = \begin{bmatrix} \max\{0.5(x_1 + x_2), 0\} \\ \max\{x_1 + x_2 - 1, 0\} \end{bmatrix}$$

Here, the **non-linearity** comes from the **rectified linear unit (ReLU)** function

$$\text{ReLU}(x) = \max\{x, 0\}$$

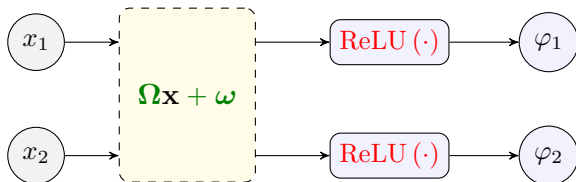


Back to the XOR Example

Defining $\text{ReLU}(\mathbf{x})$ to apply entry-wise on $\mathbf{x}$, we can write

$$\begin{aligned}\varphi(\mathbf{x}) &= \begin{bmatrix} \text{ReLU}(0.5x_1 + 0.5x_2) \\ \text{ReLU}(x_1 + x_2 - 1) \end{bmatrix} = \text{ReLU}\left(\begin{bmatrix} 0.5x_1 + 0.5x_2 \\ x_1 + x_2 - 1 \end{bmatrix}\right) \\ &= \text{ReLU}\left(\begin{bmatrix} 0.5 & 0.5 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix}\right) = \text{ReLU}(\boldsymbol{\Omega}\mathbf{x} + \boldsymbol{\omega})\end{aligned}$$

This is a **linear embedding** with only a tiny bit of **non-linearity**!



Deep Learning by Cascading Basic Blocks

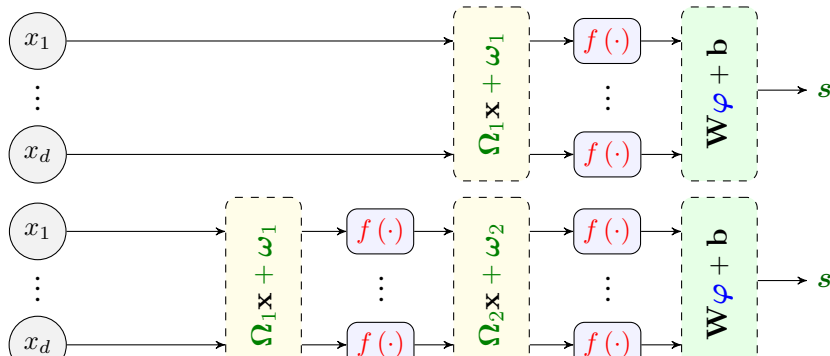
Moral of Story

Learnable linear mapping cascaded by a *nonlinear* function can be *embedding*

For simple learning problems like learning XOR function

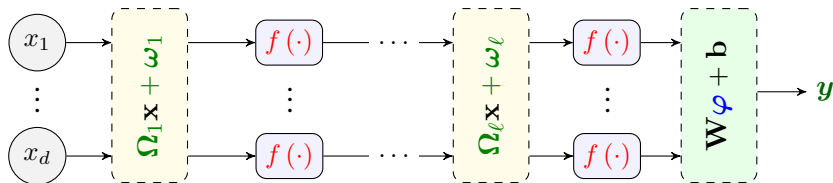
For *more complicated* problems we can *add more layers*

The *more complicated* the problem gets, the *deeper* we could go!



FNN Basics

FNN with ℓ layers between inputs and outputs looks like this:



Let's polish this diagram using some notions in the literature

Artificial Neuron

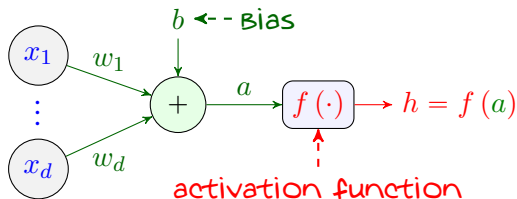
An **artificial neuron (neural unit)** is a basic computation unit with **multiple inputs** and a **single output**

that does the following items:

- It determines a **linear combination** of its inputs

$$a = \sum_{i=1}^d w_i x_i + b$$

- It sets a **function** of the **linear combination** as the output



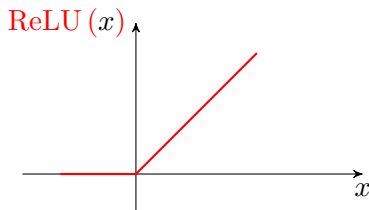
Activation

Activation function is a *nonlinear* transform $f(\cdot) : \mathbb{R} \mapsto \mathbb{R}$

We have already seen the example of **ReLU**. Other classical choices are

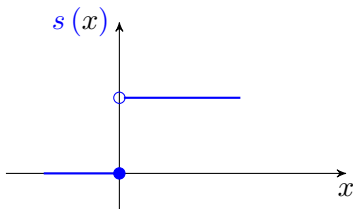
Rectified Linear Unit

$$\text{ReLU}(x) = \max\{x, 0\}$$



Step Function

$$s(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$



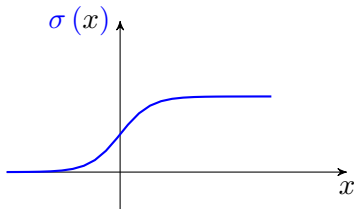
FNN Basics: Neuron

Activation function is a nonlinear transform $f(\cdot) : \mathbb{R} \mapsto \mathbb{R}$

Other classical choices are

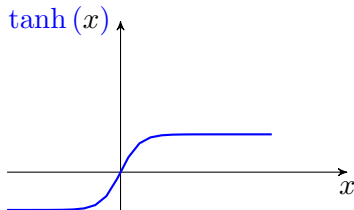
Sigmoid

$$\sigma(x) = \frac{1}{1 + \exp\{-x\}}$$



Hyperbolic Tangent

$$\tanh(x) = \frac{\exp\{x\} - \exp\{-x\}}{\exp\{x\} + \exp\{-x\}}$$



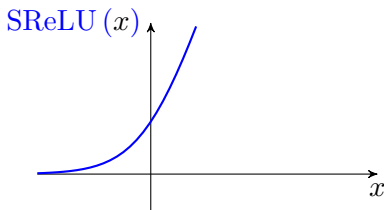
FNN Basics: Neuron

Activation function is a nonlinear transform $f(\cdot) : \mathbb{R} \mapsto \mathbb{R}$

Other classical choices are

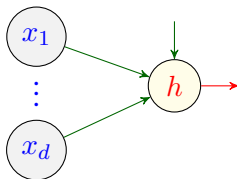
Soft ReLU

$$\text{SReLU}(x) = \log(1 + \exp\{x\})$$



Artificial Neuron

We use the following shortened diagram to represent a **neuron**



Let's make an agreement

- **green edges** are **learnable**: we find them by minimizing empirical risk
- **red edge** denotes the value we get after applying **activation**

Why do we call this block **Neuron**?

The appellation is inspired by our understanding of **biological neurons**

Further Read

- Bishop

- ↳ Chapter 5: *Section 5.1*

NNs

- ESL

- ↳ Chapter 11: *Sections 11.1 and 11.3*

NNs